



PUC Minas

Leonardo Vieira Machado

Nicolas Almeida Prado da Silva

GraphQL vs REST - Um Experimento Controlado

Belo Horizonte 2026

1	Introdução	3
1.1	Contextualização	3
1.2	Problema Foco do Experimento.....	3
1.3	Questões de Pesquisa.....	3
1.4	Hipóteses	3
1.5	Objetivos	3
2	Metodologia.....	4
2.1	Passo a Passo do Experimento	4
2.2	Decisões de Projeto.....	4
2.3	Materiais Utilizados.....	5
2.4	Métodos Utilizados	5
2.5	Métricas e Suas Unidades.....	5
2.6	Consultas avaliadas:.....	5
3	Visualização dos Resultados	6
3.1	Estatísticas Descritivas — RQ1: Tempo de Resposta (ms)	6
3.2	Estatísticas Descritivas — RQ2: Tamanho do Payload (bytes).....	6
3.3	Teste t Pareado	6
3.4	Resultados por Tipo de Consulta — Tempo Médio.....	6
3.5	Resultados por Tipo de Consulta — Tamanho Médio.....	6
4	Discussão dos Resultados.....	7
4.1	Confronto com as Questões de Pesquisa.....	7
4.2	Insights.....	7
4.3	Comparações entre Consultas	7
4.4	Estatísticas Relevantes	8
5	Conclusão	8
5.1	Tomada de Decisão.....	8
5.2	Sugestões Futuras	8
5.3	Resultado Conclusivo Sucinto	9
5.4	Confronto com a Literatura	9

1 Introdução

1.1 Contextualização

A forma como sistemas de software expõem e consomem dados por meio de APIs evoluiu significativamente nas últimas décadas. Durante muito tempo, o estilo arquitetural REST, proposto por Roy Fielding em 2000, dominou o desenvolvimento de APIs Web por sua simplicidade, aderência ao protocolo HTTP e facilidade de adoção.

Em 2015, o Facebook introduziu o GraphQL, uma linguagem de consulta para APIs que permite ao cliente especificar exatamente quais dados deseja receber, eliminando o problema de *overfetching* e *underfetching*. Em vez de endpoints fixos, GraphQL expõe um único endpoint e processa consultas estruturadas em formato de grafo.

Embora GraphQL tenha ganhado ampla adoção em produtos como GitHub, Twitter e Shopify, a literatura ainda carece de estudos empíricos rigorosos que comparem objetivamente o desempenho das duas abordagens em cenários controlados e representativos de casos de uso reais. Este trabalho busca preencher essa lacuna por meio de um experimento controlado.

1.2 Problema Foco do Experimento

O problema central investigado é: GraphQL oferece vantagem mensurável em desempenho em relação a REST? Mais especificamente, avaliam-se duas dimensões de desempenho: o tempo de resposta das APIs e o tamanho do payload retornado. Ambas as dimensões impactam diretamente a experiência do usuário e o consumo de banda, especialmente em dispositivos móveis ou redes com limitações de largura de banda. A comparação é realizada em um ambiente controlado, com a mesma base de dados, o mesmo hardware, as mesmas consultas equivalentes implementadas nas duas abordagens e o mesmo conjunto de parâmetros de entrada, de modo a isolar a variável independente: o protocolo de API utilizado (REST ou GraphQL).

1.3 Questões de Pesquisa

Este experimento é guiado por duas questões de pesquisa:

- RQ1: O tempo de resposta de APIs GraphQL é significativamente diferente do tempo de resposta de APIs REST para as mesmas consultas?
- RQ2: O tamanho do payload retornado por APIs GraphQL é significativamente diferente do tamanho do payload retornado por APIs REST para as mesmas consultas?

1.4 Hipóteses

Para cada questão de pesquisa, foram formuladas as hipóteses nula e alternativa:

RQ1 — Tempo de Resposta:

- H_0 : Não há diferença estatisticamente significativa no tempo de resposta entre GraphQL e REST.
- H_1 : Há diferença estatisticamente significativa no tempo de resposta entre GraphQL e REST.

RQ2 — Tamanho do Payload:

- H_0 : Não há diferença estatisticamente significativa no tamanho do payload entre GraphQL e REST.
- H_1 : Há diferença estatisticamente significativa no tamanho do payload entre GraphQL e REST.

O nível de significância adotado é $\alpha = 0,05$.

1.5 Objetivos

Objetivo principal: Comparar empiricamente o desempenho de APIs implementadas com GraphQL e REST,



PUC Minas

nas dimensões de tempo de resposta e tamanho de payload, por meio de um experimento controlado.

Objetivos específicos:

1. Implementar servidores REST e GraphQL funcionalmente equivalentes, atendendo ao mesmo banco de dados.
2. Definir e executar consultas equivalentes nas duas abordagens, cobrindo diferentes padrões de acesso a dados.
3. Coletar medições pareadas de tempo (ms) e tamanho (bytes) para cada par de requisições.
4. Aplicar teste estatístico pareado (teste t de Student) para determinar significância das diferenças observadas.
5. Analisar o comportamento por tipo de consulta para identificar cenários em que cada abordagem se sobressai.

2 Metodologia

2.1 Passo a Passo do Experimento

O experimento foi conduzido nas seguintes etapas:

1. Inicialização da base de dados: criação de um banco SQLite com dados sintéticos representando usuários, posts e comentários de uma rede social, utilizando a biblioteca `@faker-js/faker`.
2. Implementação dos servidores: desenvolvimento de dois servidores Node.js funcionalmente equivalentes — um REST (porta 3000) e um GraphQL (porta 3001) — conectados à mesma base de dados.
3. Definição das consultas: especificação de 5 consultas equivalentes, cada uma implementada nas duas abordagens, cobrindo diferentes padrões de complexidade e volume de dados.
4. Aquecimento: execução de 5 requisições de aquecimento em cada servidor antes do início das medições formais, para eliminar latências de inicialização de conexão e cache frio.
5. Geração e randomização dos cenários: geração de 150 cenários de teste (30 repetições $\times$ 5 tipos de consulta), com randomização da ordem de execução via embaralhamento de Fisher-Yates para evitar viés de ordenação.
6. Execução das medições: para cada cenário, medição sequencial da requisição REST e GraphQL com intervalo de 100 ms entre elas para evitar sobrecarga.
7. Persistência dos resultados: salvamento de todas as medições em resultados/medicoes.json com timestamp, identificador da consulta, parâmetro utilizado e os campos tempo_ms, tamanho_bytes e status para cada protocolo.
8. Análise estatística: execução do script analisar-resultados.js, que calcula médias, medianas, desvios padrão, percentis P95 e P99, e aplica o teste t pareado para ambas as métricas.

2.2 Decisões de Projeto

Decisão	Justificativa
Banco de dados SQLite local	Elimina variáveis de rede externas; isola o desempenho do protocolo
Mesmo banco para ambos os servidores	Garante que as respostas sejam equivalentes em conteúdo
30 repetições por consulta	Número suficiente para aplicação do Teorema Central do Limite
Randomização da ordem de execução	Evita viés temporal (ex.: aquecimento de cache favorecendo o segundo protocolo)
Delay de 100 ms entre requisições	Evita sobrecarregar o servidor e contaminar medições subsequentes
Teste t pareado	Os dados são naturalmente pareados (mesma consulta, mesmo parâmetro, medidos em sequência)
Tamanho medido via <code>JSON.stringify().length</code>	Medição do payload efetivo serializado, independente de headers HTTP

2.3 Materiais Utilizados

Hardware: computador pessoal com sistema operacional Windows, processador x86-64, execução local (sem rede externa).

Software e bibliotecas:

Componente	Versão	Função
Node.js	—	Ambiente de execução
Express	^4.18.2	Framework HTTP para servidor REST
express-graphql	^0.12.0	Middleware HTTP para servidor GraphQL
graphql	^15.8.0	Engine GraphQL (parsing, validação, execução)
sqlite3 + sqlite	^5.1.6 / ^5.1.1	Interface com banco de dados SQLite
axios	^1.6.2	Cliente HTTP para execução das requisições de teste
@faker-js/faker	^8.3.1	Geração de dados sintéticos
process.hrtime.bigint()	nativa	Medição de tempo de alta resolução (nanosegundos)

Base de dados: banco SQLite com tabelas usuarios, posts e comentários, populado com dados sintéticos. IDs de usuários testados: {1, 42, 87, 123, 256, 389, 410, 450, 487, 500}. IDs de posts testados: {100, 500, 1000, 2000, 3000, 4000, 4500, 4800, 4950, 4999}.

2.4 Métodos Utilizados

Método experimental: experimento controlado com um único fator (protocolo de API: REST vs GraphQL), variável dependente dupla (tempo de resposta e tamanho de payload) e design de medidas repetidas com dados pareados.

Coleta de dados: cada medição utiliza process.hrtime.bigint() para capturar o instante imediatamente antes e após a chamada axios(), convertendo a diferença de nanosegundos para milissegundos com 4 casas decimais de precisão.

Análise estatística: teste *t* de Student para amostras pareadas, com nível de significância $\alpha = 0,05$. A escolha do teste *t* pareado é justificada pelo design experimental: cada observação REST tem sua contraparte GraphQL exata (mesma consulta, mesmo ID de parâmetro, executada em sequência imediata).

2.5 Métricas e Suas Unidades

Métrica	Unidade	Descrição
Tempo de resposta	milissegundos (ms)	Tempo decorrido desde o envio da requisição até o recebimento completo da resposta
Tamanho do payload	bytes (B)	Comprimento da serialização JSON do corpo da resposta (JSON.stringify(response.data).length)

2.6 Consultas avaliadas:

ID	Nome	Padrão de acesso
Q1	Dados básicos de usuário	Leitura simples — 1 registro
Q2	Posts de usuário sem comentários	Leitura de lista — até 20 registros
Q3	Posts de usuário com comentários	Leitura aninhada — posts + comentários (N+1 no REST)
Q4	Dashboard completo do usuário	Consulta agregada — múltiplas sub-consultas
Q5	Comentários de um post específico	Leitura com join — comentários + dados do autor

3 Visualização dos Resultados

3.1 Estatísticas Descritivas — RQ1: Tempo de Resposta (ms)

Protocolo	Média	Mediana	Desvio Padrão	P95	P99
REST	2,83 ms	2,37 ms	1,47 ms	5,87 ms	7,32 ms
GraphQL	3,49 ms	2,93 ms	2,01 ms	8,55 ms	11,28 ms
Diferença	+0,66 ms	+0,56 ms	—	+2,68 ms	+3,96 ms
Variação %	GraphQL 23,5% mais lento	—	—	—	—

3.2 Estatísticas Descritivas — RQ2: Tamanho do Payload (bytes)

Protocolo	Média	Mediana	Máximo
REST	2.644,78 B ($\approx$ 2,58 KB)	782 B	11.717 B ($\approx$ 11,44 KB)
GraphQL	1.478,47 B ($\approx$ 1,44 KB)	824 B	7.276 B ($\approx$ 7,11 KB)
Diferença	-1.166,31 B	+42 B	-4.441 B
Variação %	GraphQL 44,1% menor	—	GraphQL 37,9% menor

Nota sobre a mediana: A mediana do tamanho é levemente maior no GraphQL (824 B vs 782 B). Isso ocorre porque as consultas simples (Q1, Q5) geram payloads pequenos que são ligeiramente maiores no GraphQL devido ao *overhead* do envelope `{"data": {...}}`. O efeito oposto domina nas consultas complexas (Q2, Q3), que são muito menores no GraphQL por não conter campos desnecessários.

3.3 Teste t Pareado

Questão	t-statistic	p-value	Significativo ($\alpha = 0,05$)
RQ1 (Tempo)	-4,1939	< 0,01	SIM ✓
RQ2 (Tamanho)	9,2354	< 0,01	SIM ✓

3.4 Resultados por Tipo de Consulta — Tempo Médio

Consulta	REST (ms)	GraphQL (ms)	Variação
Q1 — Dados básicos de usuário	1,95	2,64	REST 35,8% mais rápido
Q2 — Posts sem comentários	2,61	3,06	REST 17,4% mais rápido
Q3 — Posts com comentários	4,25	4,67	REST 9,8% mais rápido
Q4 — Dashboard completo	2,93	4,16	REST 42,1% mais rápido
Q5 — Comentários de post	2,40	2,92	REST 21,5% mais rápido

3.5 Resultados por Tipo de Consulta — Tamanho Médio

Consulta	REST	GraphQL	Variação
Q1 — Dados básicos de usuário	156 B	136 B	GraphQL 12,8% menor
Q2 — Posts sem comentários	3,95 KB	1,23 KB	GraphQL 69,0% menor
Q3 — Posts com comentários	7,65	4,58 KB	GraphQL 40,1% menor

	KB		
Q4 — Dashboard completo	781 B	823 B	REST 5,4% menor
Q5 — Comentários de post	408 B	486 B	REST 19,1% menor

4 Discussão dos Resultados

4.1 Confronto com as Questões de Pesquisa

RQ1 — O tempo de resposta GraphQL é significativamente diferente do REST?

Sim. O teste t pareado rejeitou H_0 com $p\text{-value} < 0,01$ ($t = -4,1939$). REST apresentou tempo médio 23,5% menor que GraphQL (2,83 ms vs 3,49 ms). Este resultado foi consistente em todos os 5 tipos de consulta testados: o REST foi mais rápido em todas elas.

RQ2 — O tamanho do payload GraphQL é significativamente diferente do REST?

Sim. O teste t pareado rejeitou H_0 com $p\text{-value} < 0,01$ ($t = 9,2354$). GraphQL apresentou payload médio 44,1% menor que REST (1.478 B vs 2.644 B). Porém, os resultados variam substancialmente por tipo de consulta: GraphQL foi menor em Q1, Q2 e Q3, enquanto REST foi menor em Q4 e Q5.

4.2 Insights

Insight 1 — O overhead do GraphQL em tempo é real, mas pequeno em valor absoluto. A diferença média de 0,66 ms é estatisticamente significativa, mas praticamente irrelevante em aplicações reais onde latências de rede são ordens de magnitude maiores (dezenas a centenas de ms). Em ambiente de produção com rede, esse overhead seria diluído no ruído de comunicação.

Insight 2 — GraphQL entrega seu maior benefício em consultas com dados volumosos. A consulta Q2 (posts sem comentários) registrou redução de 69,0% no payload, REST retornou 3,95 KB enquanto GraphQL retornou apenas 1,23 KB, porque a API REST retorna todos os campos do schema (incluindo conteúdo, campo de texto longo), enquanto a query GraphQL solicitou apenas id, título, curtidas e data_post.

Insight 3. Consultas simples favorecem REST no tamanho. Para Q4 e Q5, REST gerou payloads menores. Isso ocorre porque o GraphQL adiciona um *envelope* {"data": {"queryName": [...]} em toda resposta, que representa overhead fixo proporcional ao número de campos. Quando o payload é pequeno (poucos campos, poucos registros), esse overhead pode superar o ganho do *selective fetching*.

Insight 4 — GraphQL apresenta maior variabilidade no tempo de resposta. O desvio padrão do GraphQL (2,01 ms) é 37% maior que o do REST (1,47 ms), e os percentis P95 e P99 do GraphQL são significativamente mais altos (8,55 ms e 11,28 ms vs 5,87 ms e 7,32 ms). Isso sugere que o motor de execução GraphQL (parsing, validação e resolução do grafo de consulta) introduz latência variável que não está presente no roteamento direto REST.

Insight 5 — O problema N+1 foi mitigado, mas não eliminado, no GraphQL. A consulta Q3 (posts com comentários) registrou 40,1% menos bytes no GraphQL, indicando que o *selective fetching* funcionou. Contudo, a implementação dos resolvers de posts e comentários no GraphQL ainda executa queries sequenciais, não tendo implementado *DataLoader* para batching. Em uma implementação otimizada com *DataLoader*, o GraphQL provavelmente apresentaria tempo de resposta ainda menor neste caso.

4.3 Comparações entre Consultas

Cenário	Vencedor em Tempo	Vencedor em Tamanho	Recomendação
Consultas simples, 1 objeto	REST	GraphQL (pequena margem)	REST para tempo crítico



PUC Minas

Listas com seleção de campos	REST	GraphQL (grande vantagem)	GraphQL para mobile/banda limitada
Consultas aninhadas	REST	GraphQL	Depende da prioridade
Dados agregados	REST	REST	REST em ambas as dimensões
Consultas com join	REST	REST	REST em ambas as dimensões

4.4 Estatísticas Relevantes

- Total de medições: 150 (30 repetições × 5 consultas)
- Taxa de sucesso: 100% — todas as requisições retornaram HTTP 200 sem erros
- Ganho REST em tempo: consistente nos 5 tipos de consulta (100% das comparações)
- Ganho GraphQL em tamanho: presente em 3 dos 5 tipos de consulta (60% das comparações)
- Redução máxima de payload GraphQL: 69,0% na consulta Q2
- Overhead máximo de tempo GraphQL: 42,1% na consulta Q4

5 Conclusão

5.1 Tomada de Decisão

A análise dos dados experimentais indica que não existe um "vencedor absoluto" entre GraphQL e REST: as duas abordagens apresentam vantagens em dimensões distintas.

- REST vence em tempo de resposta em todos os 5 cenários testados, com diferença estatisticamente significativa ($p < 0,01$). O overhead do motor GraphQL (parsing de query, validação de schema, resolução do grafo) é mensurável e consistente.
- GraphQL vence em tamanho de payload em consultas de média e alta complexidade que envolvem listas de registros ou dados aninhados, com reduções de até 69%. Para consultas simples ou com poucos campos, o overhead do envelope JSON do GraphQL pode fazer o payload ser ligeiramente maior que o REST equivalente.

Recomendação prática: para aplicações que priorizam baixa latência (sistemas de trading, dashboards em tempo real, APIs internas de alta frequência), REST é a escolha mais adequada. Para aplicações que priorizam eficiência de banda (aplicativos móveis, APIs públicas consumidas por clientes variados, sistemas com múltiplos front-ends com necessidades distintas), GraphQL apresenta vantagem significativa, especialmente quando o *overfetching* do REST seria pronunciado.

5.2 Sugestões Futuras

1. Implementar DataLoader no GraphQL: o padrão de resolvers atual não usa batching, o que resulta em múltiplas queries ao banco para relações 1:N. A adoção de DataLoader poderia reduzir significativamente o tempo das consultas aninhadas (Q3), possivelmente invertendo o resultado de RQ1 para esse tipo de consulta.
2. Avaliar com rede real: os experimentos foram conduzidos localmente (loopback). Em redes reais com latências de 20–200 ms, a redução de tamanho de payload do GraphQL se tornaria ainda mais vantajosa (menos bytes = menor tempo de transmissão), enquanto o overhead de tempo do motor GraphQL seria proporcionalmente menos relevante.
3. Aumentar a amostra: 30 repetições por consulta são suficientes para testes de significância, mas um n maior permitiria análises de distribuição mais robustas e detecção de outliers com maior precisão.
4. Incluir consultas de escrita (mutations): este experimento avaliou exclusivamente operações de leitura. Mutações GraphQL e endpoints POST/PUT/DELETE REST apresentam características de desempenho distintas que merecem investigação própria.
5. Avaliar com caching: tanto REST (via HTTP caching com ETags/Last-Modified) quanto GraphQL (via caching por persisted queries) possuem estratégias de cache. Um experimento comparando o desempenho com caching habilitado poderia revelar diferenças significativas.
6. Testar com subscrições GraphQL: as *subscriptions* são uma funcionalidade exclusiva do GraphQL (sem



PUC Minas

equivalente nativo em REST) e representam um caso de uso adicional que justificaria a adoção do protocolo.

5.3 Resultado Conclusivo Sucinto

Com 150 medições, o experimento demonstrou que:

- REST é 23,5% mais rápido em tempo de resposta ($p < 0,01$)
- GraphQL gera payloads 44,1% menores em média ($p < 0,01$)
- Ambas as diferenças são estatisticamente significativas

A escolha entre os dois deve ser guiada pelas restrições do projeto: se tempo é o fator crítico, use REST; se eficiência de banda é o fator crítico (e as consultas envolvem dados volumosos), use GraphQL.

5.4 Confronto com a Literatura

Os resultados deste experimento estão alinhados com achados de trabalhos anteriores na área:

- Brito et al. (2019): "REST vs GraphQL: A Controlled Experiment" — encontraram que GraphQL reduz o número de requisições e o volume de dados transferidos, mas adiciona overhead de processamento no servidor. Os autores concluem que GraphQL é mais adequado para cenários com clientes heterogêneos, o que corrobora os resultados deste trabalho.
- Vargas et al. (2018): Ao comparar REST e GraphQL em aplicações móveis, observaram que a redução de payload do GraphQL resulta em economia real de bateria e dados do dispositivo, especialmente em conexões 4G — contexto em que a diferença de 44,1% no tamanho encontrada neste experimento seria diretamente impactante.
- Hartig & Pérez (2018): Em análise formal da semântica do GraphQL, os autores apontam que a complexidade de execução das queries GraphQL é potencialmente maior que a das consultas REST equivalentes, o que é consistente com o overhead de tempo observado (23,5%) neste estudo.
- GitHub GraphQL API (documentação oficial) — O GitHub relata que, ao migrar sua API pública para GraphQL v4, clientes conseguiram reduzir em até 50% o volume de dados trafegados em comparação com a API REST v3, número compatível com a redução de 44,1% observada neste experimento.

A principal contradição em relação à expectativa inicial é que GraphQL não apresentou vantagem em tempo de resposta mesmo em cenários que teoricamente eliminam o problema N+1 (Q3 — posts com comentários). A hipótese para explicar isso é que a implementação dos resolvers neste experimento ainda executa múltiplas queries sequenciais ao banco, sem o padrão *DataLoader*, o que anula o potencial ganho de execução paralela que GraphQL poderia oferecer.